Horizon Campus — Faculty of Information Technology

IT41043 — Intelligent Systems (Agentic AI)

Assignment Brief: Designing, Building & Deploying an Agentic AI System

Module	IT41043 — Intelligent Systems / Agentic AI	**Weighting**
Mode	Individual (or pairs, at lecturer's discretion)	10% of module
Submission	GitHub repository link + live Streamlit Cloud URL, via LMS	Deadline: 27th July 2026

1. Overview

You will design, build, and deploy a working Agentic AI application that either solves a real-world problem or supports a research project you are actively conducting. The application must demonstrate applied understanding of agentic AI design patterns, agent-to-agent communication, deliberate model selection across OpenRouter or Groq, and a Retrieval-Augmented Generation (RAG) component grounded in a domain-specific knowledge base.

This is not a tutorial-replication exercise. Two submissions built from the same generic prompt to an AI assistant should look nothing alike, because your topic, data, and design decisions are your own.

2. Learning Outcomes Addressed

- Critically evaluate and apply Agentic AI design patterns
- Design and implement agent-to-agent communication
- Justify model selection for a task based on cost, latency, context length, and reasoning ability
- Build and integrate a Retrieval-Augmented Generation pipeline
- Deploy and demonstrate a working, publicly accessible AI system
- Apply professional software engineering and version-control practice

3. Task Description

Choose ONE of the following project types.

- **Option A**: Real-world problem: an agentic AI application addressing a genuine problem for an organisation, community, or sector (e.g. a specific Sri Lankan SME, university service, healthcare, agriculture, or public-service scenario).
- **Option B**: Research support tool: an agentic AI application that supports research you are currently conducting (e.g. literature triage, data collection/annotation assistance, structural analysis of a dataset).

Your registered topic, dataset, and architecture decisions must be your own. Generic "customer support chatbot" or "PDF Q&A bot" submissions with no domain specificity will be capped at a pass grade regardless of technical execution.

4. Technical Requirements (mandatory)

a) Agentic design patterns

Implement at least three distinct patterns, e.g. ReAct, planning/task-decomposition, reflection/self-critique, tool-use, router, or orchestrator–worker. Your README must name each pattern used and where in the code it lives.

b) Agent-to-agent communication

Implement at least two distinct agents that exchange structured messages (using LangGraph, CrewAI, AutoGen, or a custom protocol inspired by MCP/A2A). Include a sequence or message-flow diagram in your README.

c) Model selection strategy

Use OpenRouter and/or Groq. Deliberately select at least two different models for different sub-tasks. Do not use one model for everything. Your README must include a short comparison table justifying each choice against latency, cost per token, context window, and reasoning quality. Example structure:

Sub-task	Model (provider)	Why chosen
Intent routing / cheap classification	e.g. Llama 3.1 8B (Groq)	Very low latency, near-free, sufficient for simple routing decisions
Deep reasoning / final synthesis	e.g. Claude or GPT-4-class (OpenRouter)	Higher reasoning quality justifies higher cost/latency for the final answer
Retrieval re-ranking	e.g. small open model (Groq)	Fast enough to score many chunks without becoming a bottleneck

d) RAG integration

Ingest a domain-specific corpus (roughly 20+ documents or equivalent) relevant to your chosen problem. Document your chunking strategy, embedding model, and vector store (e.g. Chroma, FAISS, or a free-tier hosted store). Include a short retrieval evaluation: run 5 sample queries and comment on whether the retrieved context was actually relevant.

e) Deployment

The application must be deployed and publicly reachable on Streamlit Community Cloud, and must remain live for at least two weeks after the deadline for marking.

f) Secrets management

API keys must be handled via Streamlit secrets or environment variables and must never appear in the committed source code or commit history. A .gitignore is mandatory.

5. GitHub Requirements

- Repository is public, or private with the lecturer added as a collaborator.
- README.md must include: project description, architecture diagram, setup instructions, the model-choice comparison table, the agent-communication diagram, an explanation of the RAG pipeline, the live Streamlit demo link, and known limitations.
- Feature branches: one branch per feature, e.g. feature/rag-pipeline, feature/agent-orchestration, feature/streamlit-ui, feature/model-router, merged into main via Pull Requests with descriptive titles and descriptions.
- At least 15 meaningful, incremental commits spread across the development period, a single or two-commit bulk upload does not meet this requirement and will be treated as an integrity flag (see Section 6).
- Semantic commit messages are required: feat:, fix:, docs:, refactor:, test:.
- A GitHub Issues board or Projects board tracking milestones is recommended but not mandatory.

6. Academic Integrity & Anti-Misuse Measures

Uploading this brief to a chatbot and submitting whatever comes back is an academic integrity violation under Horizon Campus policy. The following layered controls apply to this assignment:

- Live individual viva (10–15 minutes): you will walk through your architecture and make a small live modification to your own code. This is the primary defence and carries real marks (Section 8).
- Git commit-history review: commits should show organic, iterative development over time, not a single bulk push.
- Code-similarity checks across the cohort's submissions, alongside AI-content detection used as one signal among several, never as sole proof.

7. Deliverables & Submission

- GitHub repository link (main branch holding the final, merged, working version).
- Live Streamlit Community Cloud URL.

- README.md as specified in Section 5.
- Optional but recommended: a short (max 2 minute) screen-recorded demo link included in the README.md.

Submit the GitHub link via the LMS submission box before the deadline. Late submissions follow standard Horizon Campus late-penalty policy.

8. Marking Rubric

Criteria	Description	Marks
Agentic design patterns	Correct, justified use of ≥3 patterns (ReAct, planning, reflection, tool-use, router, orchestrator-worker, etc.)	15
Agent-to-agent communication	≥2 agents exchanging structured messages; clear protocol; sequence diagram in README	15
Model selection strategy	≥2 models from OpenRouter/Groq deliberately chosen per sub-task; justified with cost/latency/context/quality comparison	15
RAG integration	Working retrieval pipeline over a real domain corpus; chunking & embedding choices explained; retrieval quality evaluated	15
Streamlit deployment	App live on Streamlit Community Cloud; functional, usable, handles errors gracefully	10
GitHub practice	Feature branches, PRs, ≥15 meaningful incremental commits, clean history, working README	15
Academic integrity & viva	Student can explain and modify their own code live; originality of topic and implementation	15

Total: 100 marks, scaled to the module weighting shown above.

9. Indicative Timeline

- 30 Minutes: Topic formulation and Git & GitHub Setup with collaborators.
- 1 Hour & 30 Minutes: Architecture design and testing using Google Colab or Jupyter Notebook (architecture + at least one working agent).
- 1 Hour & 30 Minutes: Streamlit application development
- 30 Minutes: Deployment & Debugging

10. Declaration

By submitting, you confirm that the work is your own, that any external tools or libraries used are disclosed in the README, and that you can explain and modify every part of your submission on request.